

Day 1 - 2



# Transformers

Jonghyun Choi

Associate Professor

Department of Electrical and Computer Engineering  
Seoul National University

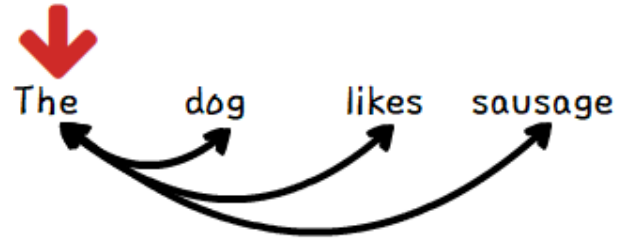


# Schedule

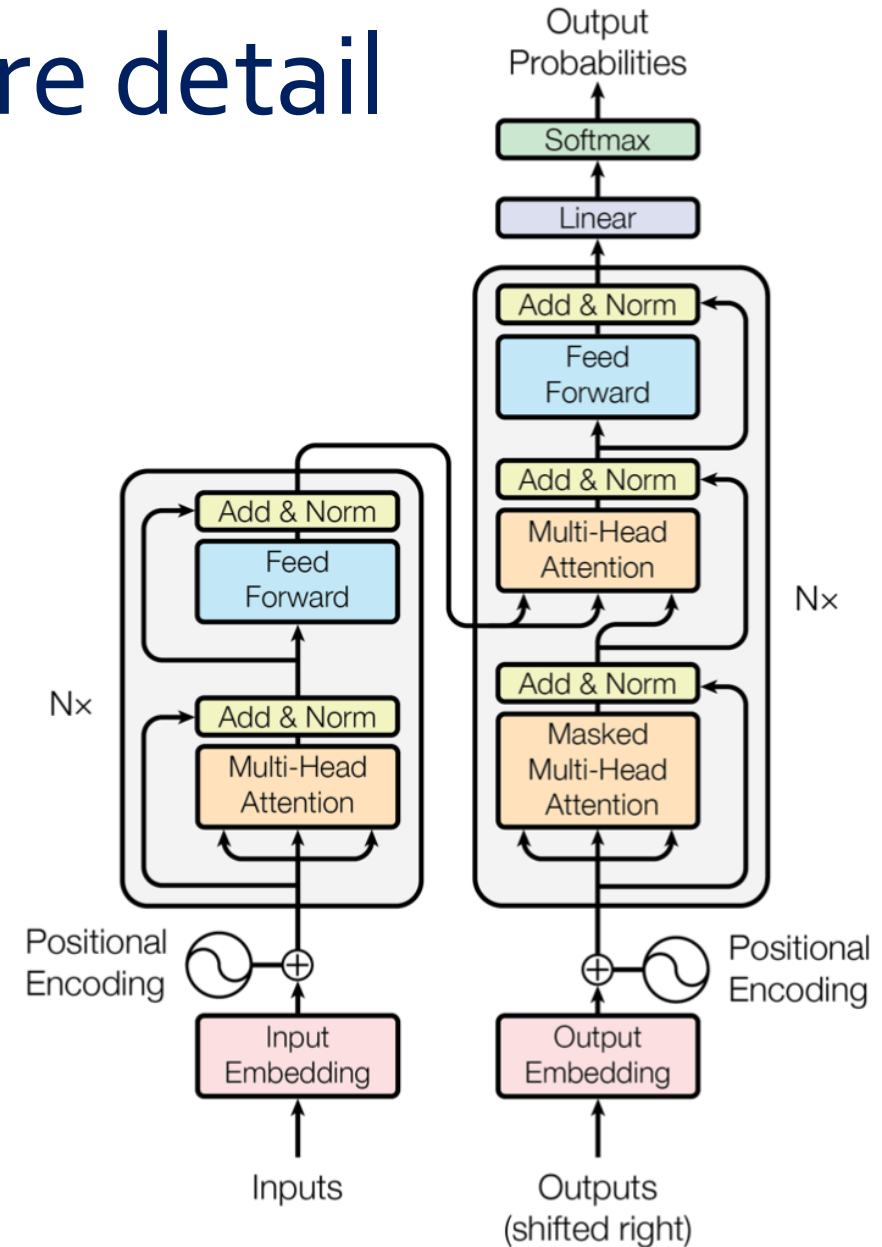
Each hour is scheduled as **50 min lecture + 10 min break**

- **Day 1:** 4 hr lecture + 4 hr practice
  - **AM:** CNN, Transformers, Object detection
  - **PM:** DETR, MI-DETR
- **Day 2:** 4 hr lecture + 4 hr practice
  - **AM:** Self-supervised learning, Weakly supervised learning, Multimodal models, Semantic segmentation
  - **PM:** DINO, SAM variants

# Transformer: A seq-to-seq model



# Transformer – Architecture detail



# Three key ideas of Transformers

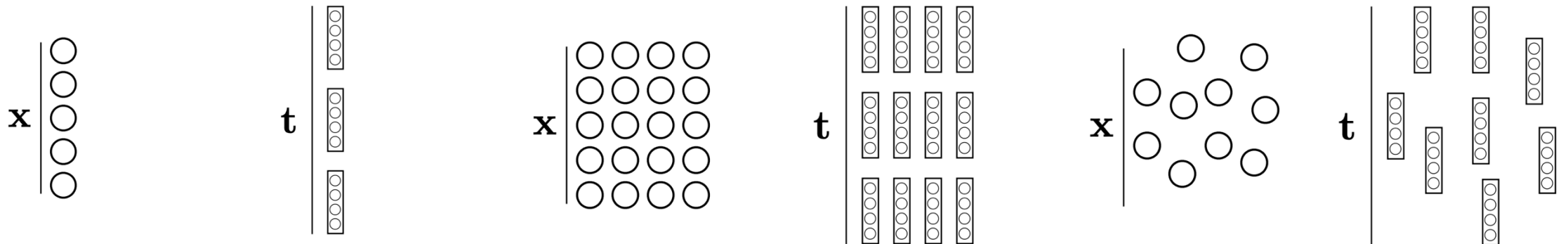
- Tokens
- Attention
- Positional encoding

# Token

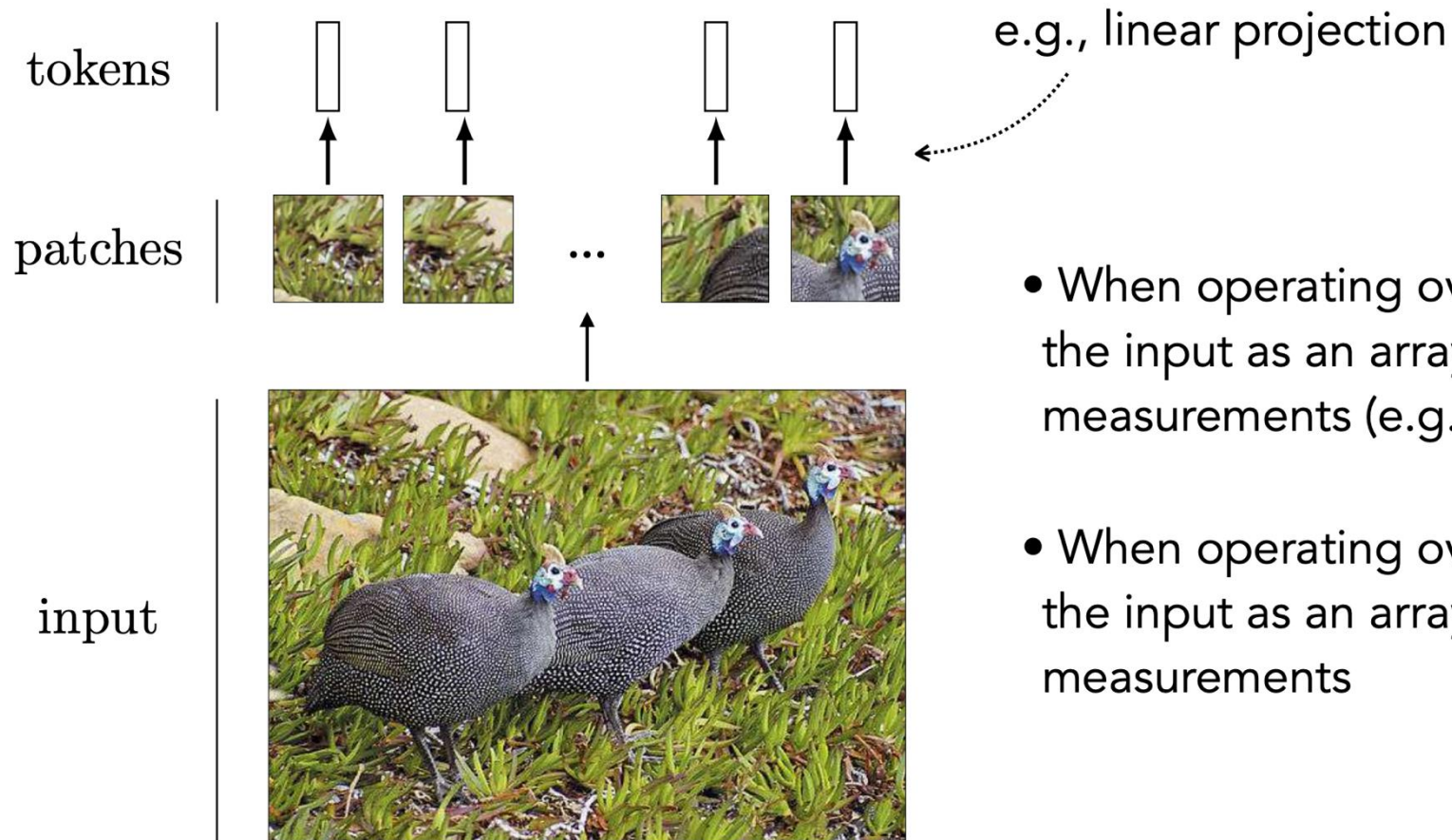
# A new data structure: Tokens

- A vector of neurons
  - **Note:** GNNs also operate over tokens, but over there we called them “node attributes” or node “feature descriptors”
- An encapsulated bundle of information
  - With transformers we will operate over tokens rather than over neurons

array of **neurons**   array of **tokens**   array of **neurons**   array of **tokens**   set of **neurons**   set of **tokens**



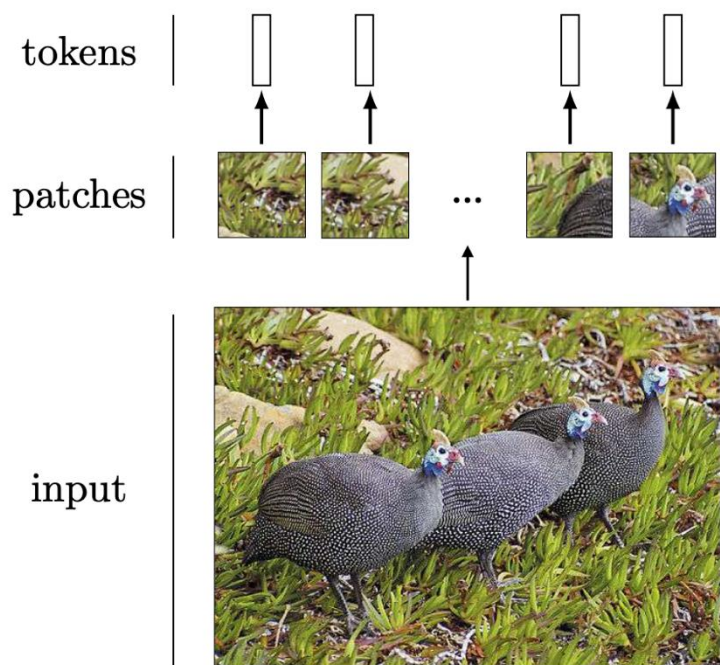
# Tokenizing the image input



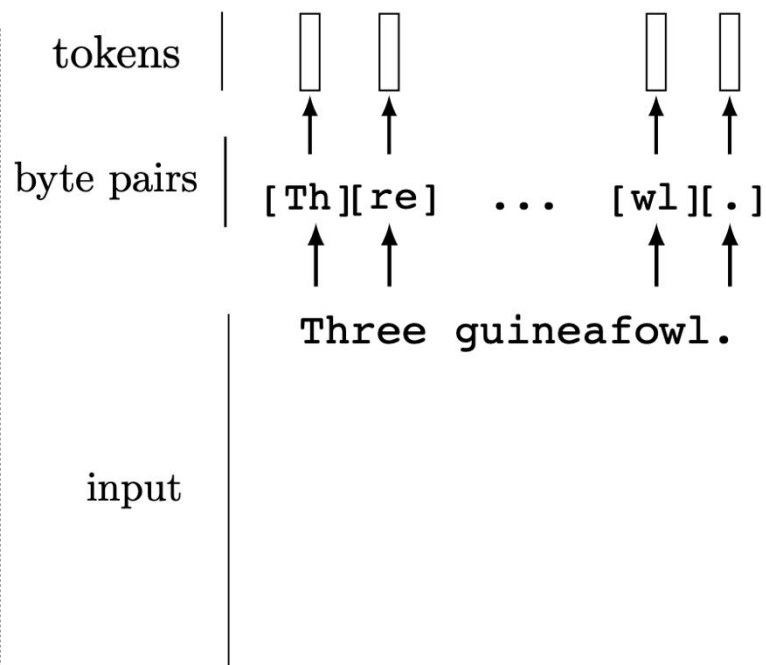
- When operating over *neurons*, we represent the input as an array of scalar-valued measurements (e.g., pixels)
- When operating over *tokens*, we represent the input as an array of vector-valued measurements

# Tokenizing different input types

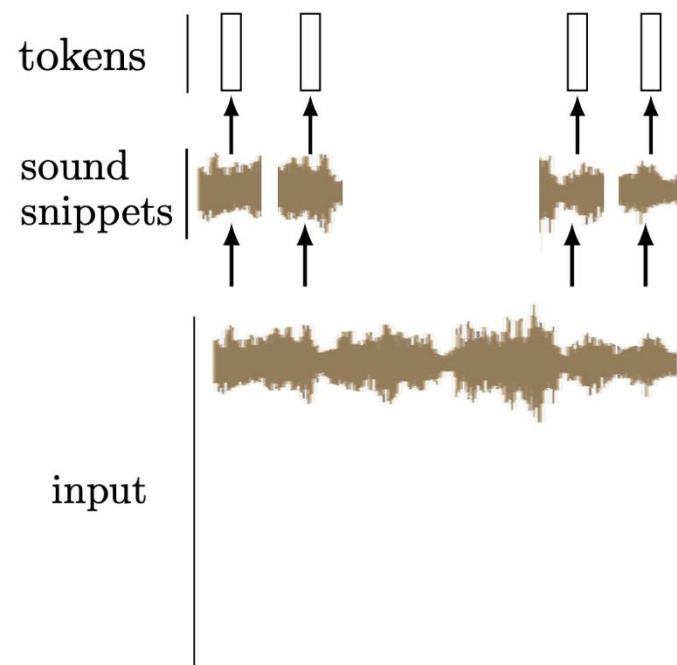
- Chop the input up into chunks, project each chunk to a vector



Image



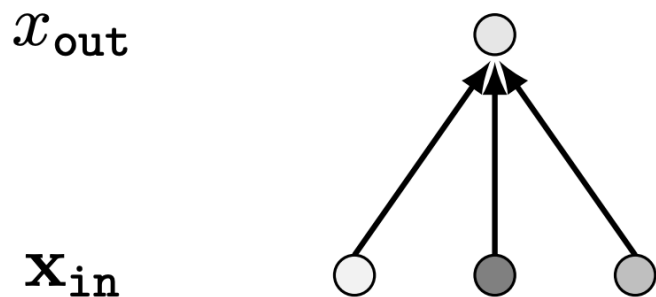
Words



Audio

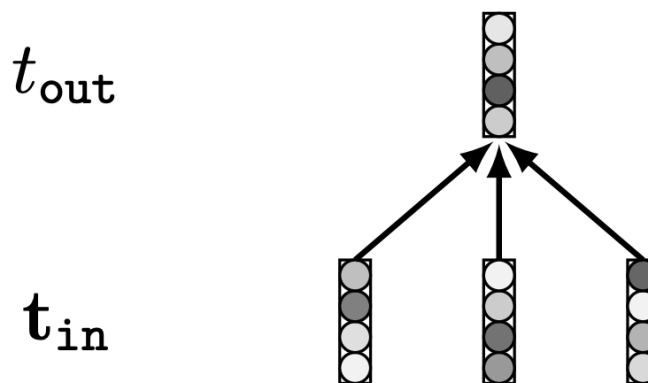
# Linear combination of tokens

Linear combination of neurons



$$x_{\text{out}} = \sum_{i=1}^N w_i x_{\text{in}_i}$$

Linear combination of tokens



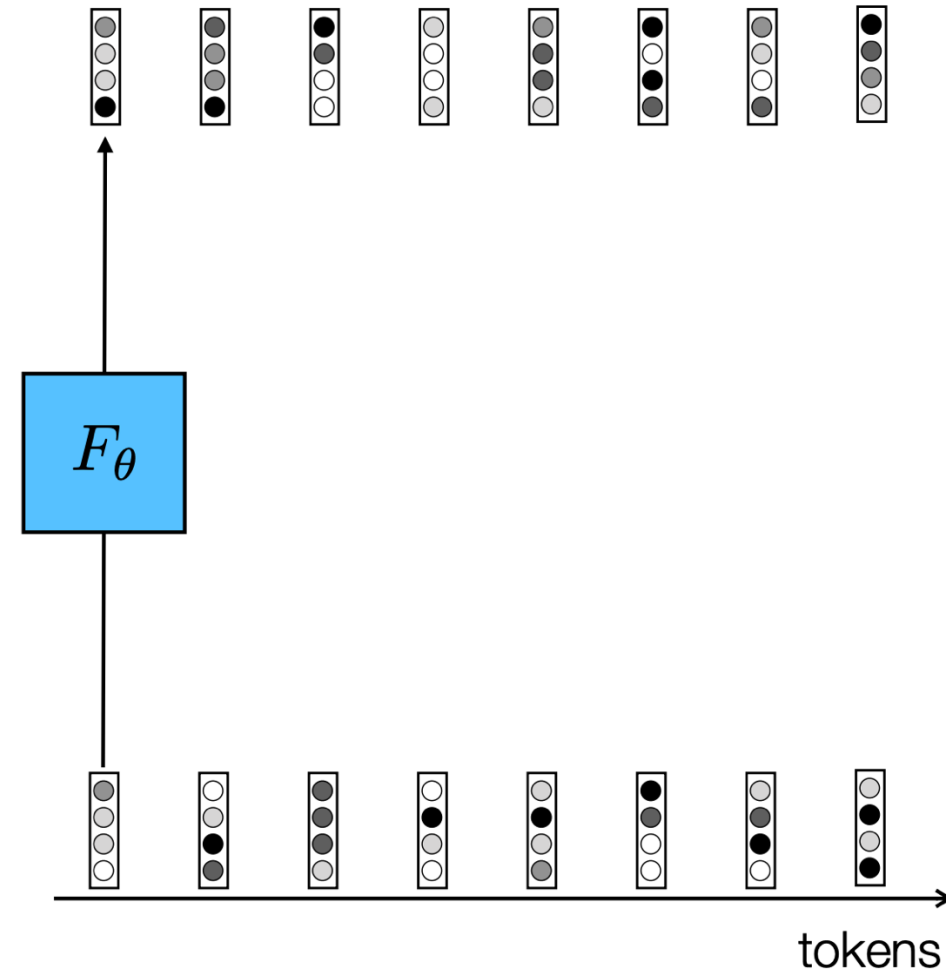
$$t_{\text{out}} \cdot \mathbf{z} = \sum_{i=1}^N w_i t_{\text{in}_i} \cdot \mathbf{z}$$

# Token-wise nonlinearity

$$\mathbf{t}_{\text{out}} = [F_{\theta}(t_1.\mathbf{z}), \dots, F_{\theta}(t_N.\mathbf{z})]$$

$F$  is typically an MLP

Equivalent to a CNN with 1x1  
kernels run over token sequence



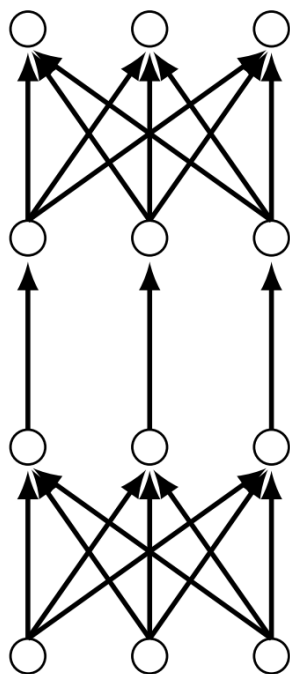
# Neural net **vs** Token net

**Neural net**

linear comb of neurons ▷

neuron-wise nonlinearity ▷

linear comb of neurons ▷

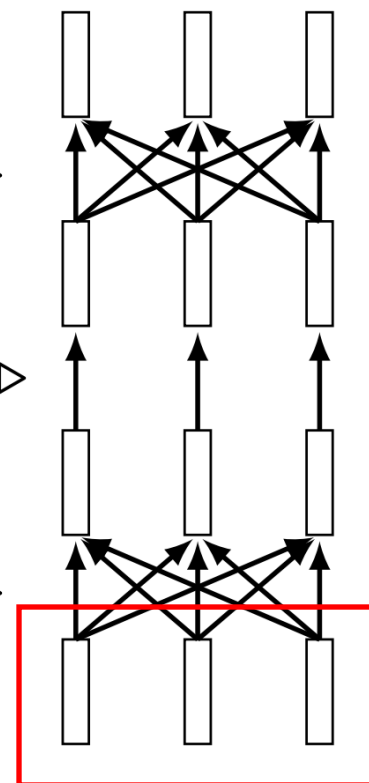


**Token net**

linear comb of tokens ▷

token-wise nonlinearity ▷

linear comb of tokens ▷



Window

# Attention

# A limitation of CNNs

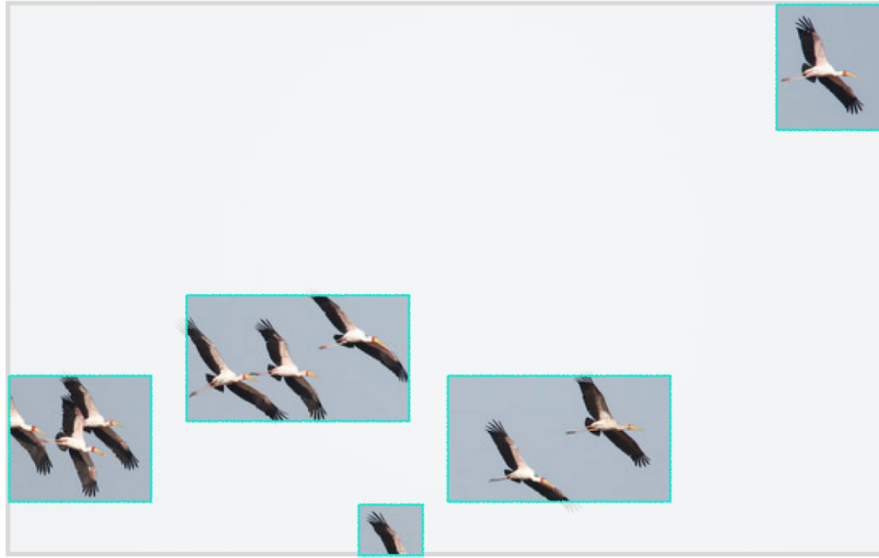


How many birds are in this image?

Is the top right bird the same species as the bottom left bird?

CNNs are built around the idea of locality, and are not well-suited to modeling long distance relationships

# A limitation of CNNs



How many birds are in this image?

# A limitation of CNNs



Is the top right bird the same species as the bottom left bird?

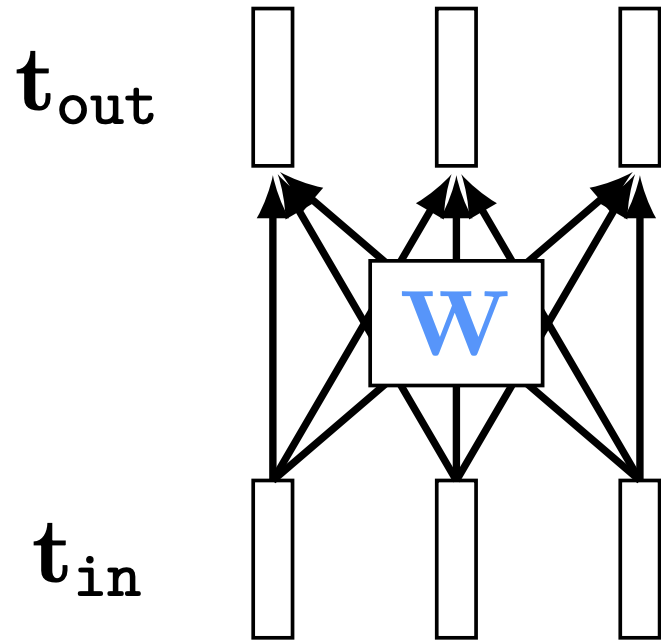
# A limitation of CNNs



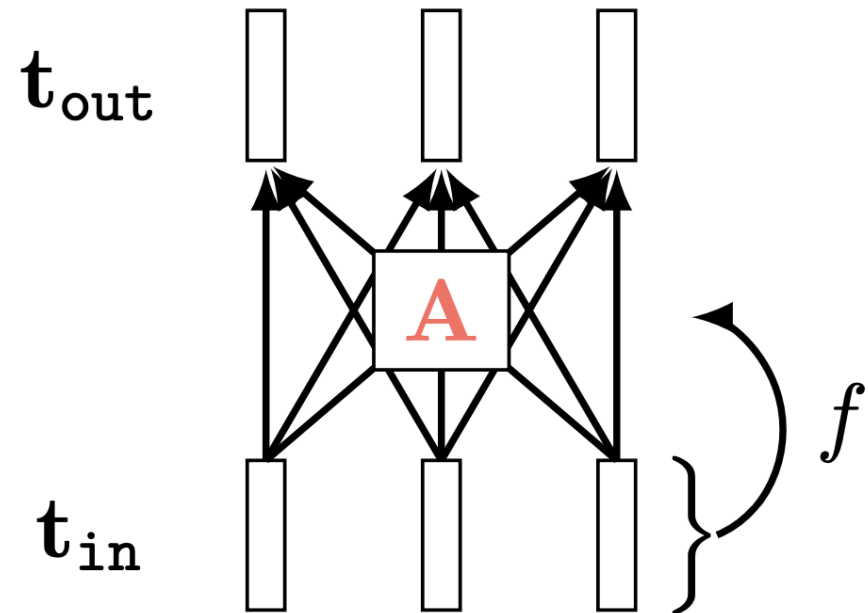
What's the color of the sky?

# (Ordinary) weight **vs** (self) attention

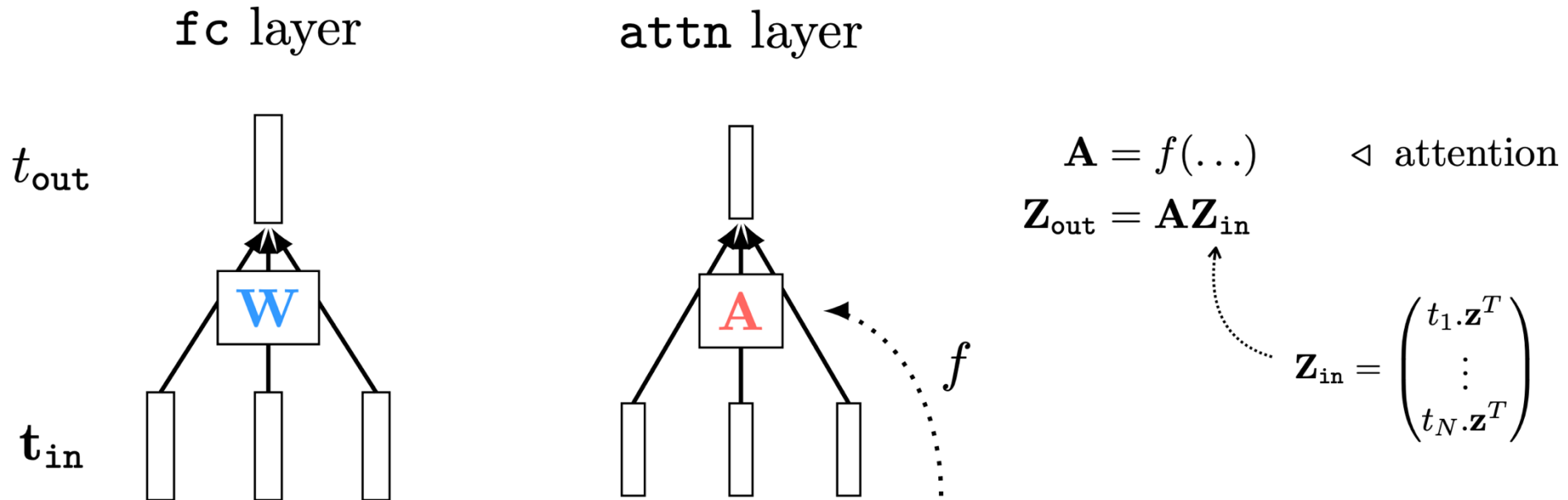
fc layer



self attn layer



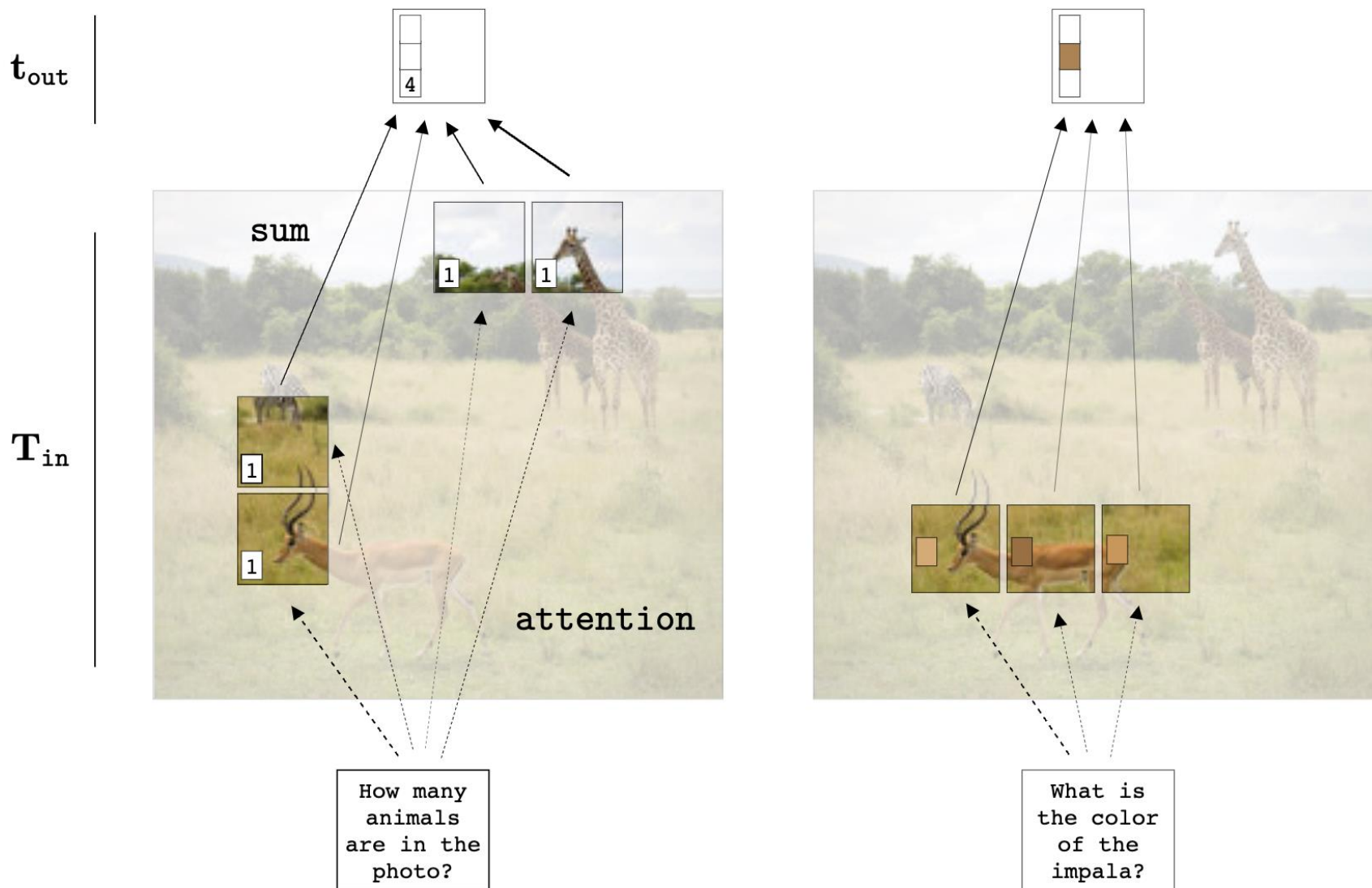
# Attention: A weighting scheme considering input



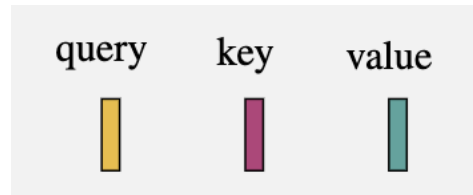
$\mathbf{W}$  is free parameters.

$\mathbf{A}$  is a function of some input data. The data tells us which tokens to attend to (assign high weight in weighted sum)

# It is a dynamic pooling operation based on query



# How does the attention work? An example of visual question answering (VQA) task



$$\mathbf{q} = \mathbf{W}_q \mathbf{t} \quad / \text{ query}$$

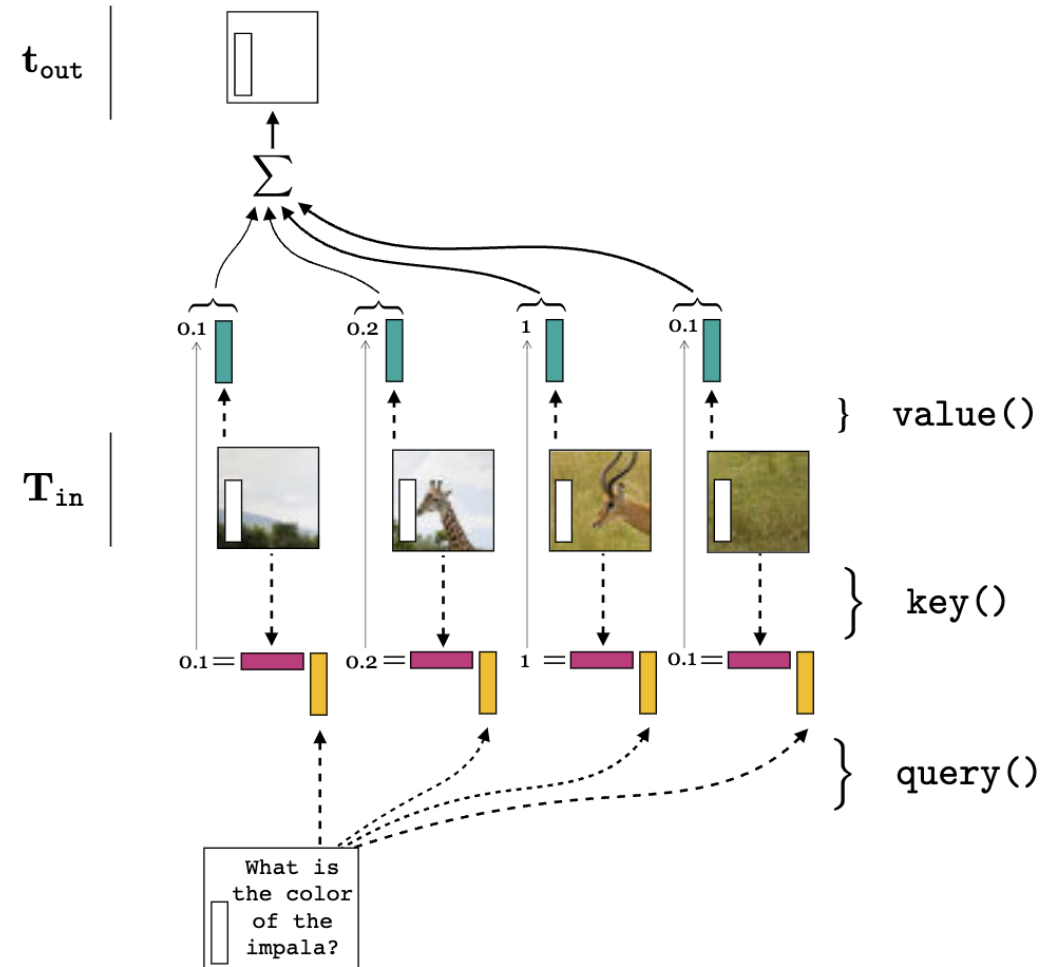
$$\mathbf{k} = \mathbf{W}_k \mathbf{t} \quad / \text{ key}$$

$$\mathbf{v} = \mathbf{W}_v \mathbf{t} \quad / \text{ value}$$

$$\mathbf{s} = [s_1, \dots, s_N]^T = [\mathbf{q}_{\text{question}}^T \mathbf{k}_1, \dots, \mathbf{q}_{\text{question}}^T \mathbf{k}_N]^T$$

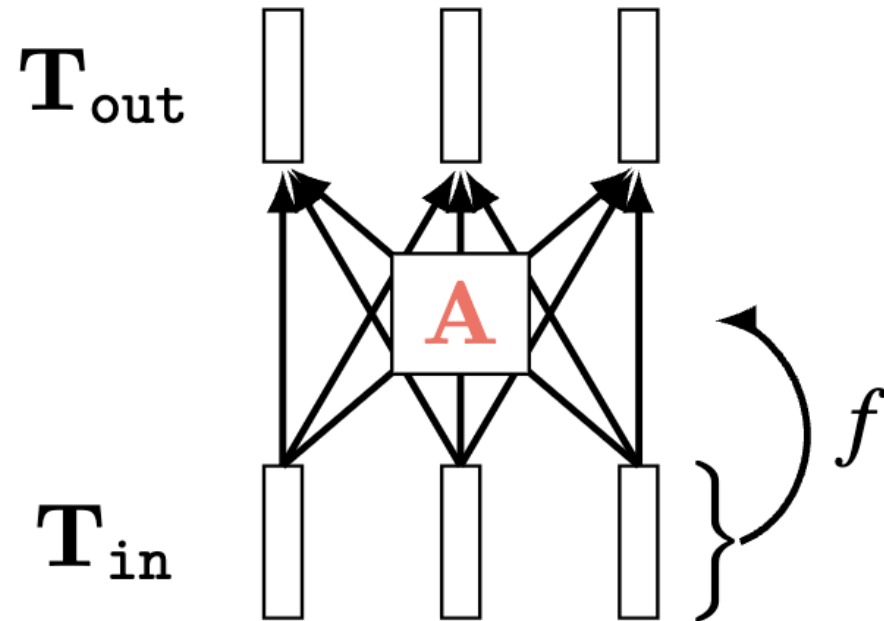
$$\mathbf{a} = \text{softmax}(\mathbf{s})$$

$$\mathbf{T}_{\text{out}} = \begin{bmatrix} a_1 \mathbf{v}_1^T \\ \vdots \\ a_N \mathbf{v}_N^T \end{bmatrix}$$



# Self-attention

self attn layer



# Self-attention in matrix operation

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top \quad / \quad \text{query matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top \quad / \quad \text{key matrix}$$

$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top \quad / \quad \text{value matrix}$$

# Self-attention in matrix operation

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top \quad / \quad \text{query matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top \quad / \quad \text{key matrix}$$

$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top \quad / \quad \text{value matrix}$$

$$\mathbf{A} = f(\mathbf{T}_{\text{in}}) = \text{softmax} \left( \frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^\top}{\sqrt{m}} \right) \quad / \quad \text{attention matrix}$$

$$\mathbf{T}_{\text{out}} = \mathbf{A} \mathbf{V}_{\text{in}}$$

# Self-attention in matrix operation

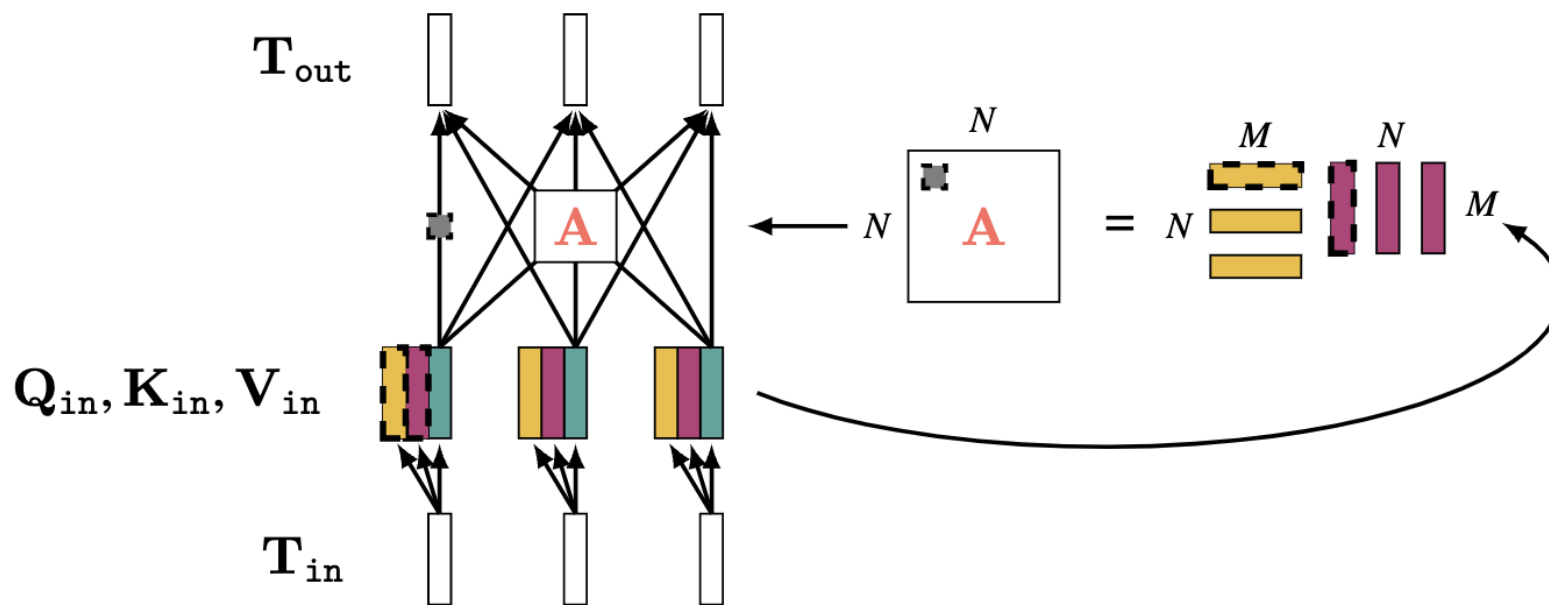
$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top \quad / \quad \text{query matrix} \quad \mathbf{A} = f(\mathbf{T}_{\text{in}}) = \text{softmax}\left(\frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^\top}{\sqrt{m}}\right) \quad / \quad \text{attention matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_n)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top \quad / \quad \text{key matrix}$$

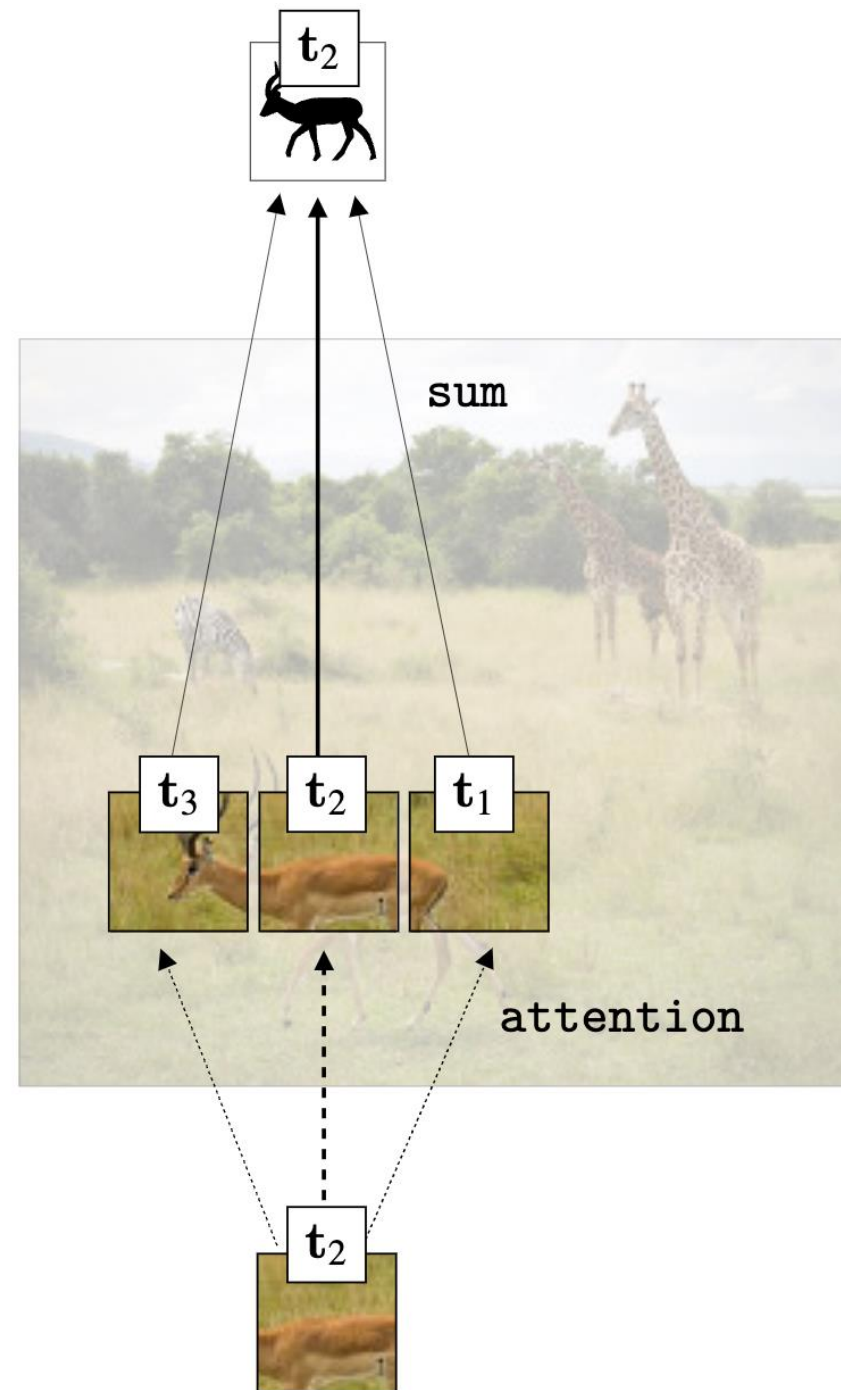
$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top \quad / \quad \text{value matrix}$$

$$\mathbf{T}_{\text{out}} = \mathbf{A} \mathbf{V}_{\text{in}}$$

self attn layer (expanded)



Self-attention is to  
refine representation of  
patch for its semantic  
considering  
neighboring patches



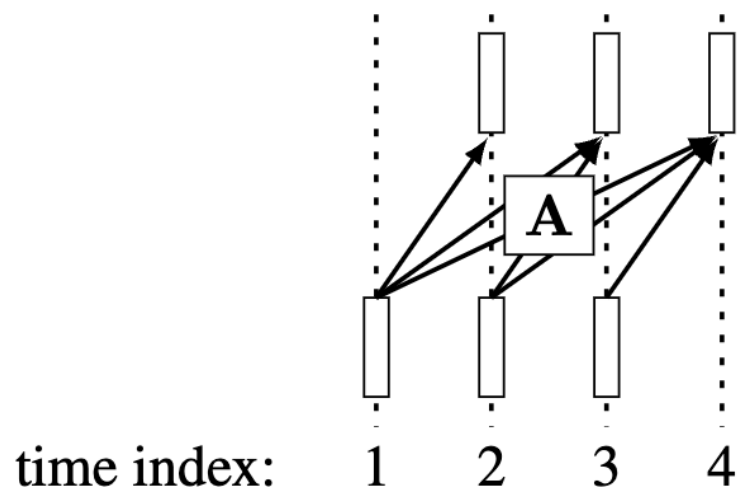
# An example of patch token value refinement by self-attention

- DINO [1]
  - Visualize the small patch to be labeled as dog



# Masked attention

- Sometimes we want to restrict which tokens can attend to which
- By masking the attention matrix, we can implement this
- **Usually used in decoder**: as we want to attend to the “previous tokens” not the “future tokens”



$$\mathbf{A} \mathbf{T}_{\text{in}} = \mathbf{T}_{\text{out}}$$

The diagram shows the matrix equation  $\mathbf{A} \mathbf{T}_{\text{in}} = \mathbf{T}_{\text{out}}$ . Matrix  $\mathbf{A}$  is a 4x4 grid where the upper triangle is black (masked) and the lower triangle is white. Vector  $\mathbf{T}_{\text{in}}$  is a 4x1 column vector with the bottom element black and others white. Vector  $\mathbf{T}_{\text{out}}$  is a 4x1 column vector with the top element black and others white. The equation  $\mathbf{A} \mathbf{T}_{\text{in}} = \mathbf{T}_{\text{out}}$  is shown.

# Masked attention can be used for selective data encoding for multimodal input [1]

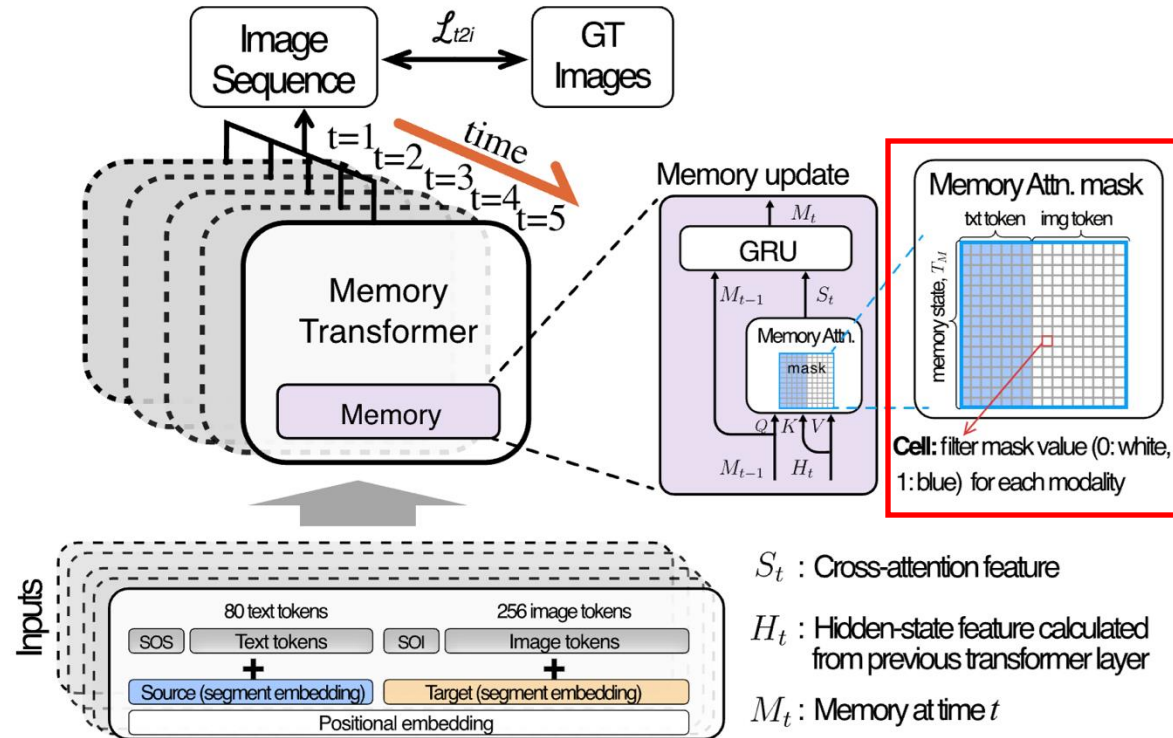


Figure 2. **Detailed input configuration for the proposed transformer.** As an input, we concatenate text and image tokens with two embeddings, *i.e.*, ‘SOS’ (start of sentence token) and ‘SOI’ (start of image token), and then add positional and segment embedding. As we described in the main paper, we use memory-attention (memory attn.) mask to selectively determine which information to be propagated as a memory represented as a blue-colored box. Here, we illustrate using the ‘text’ information as memory content.

# Masked attention can be used for selective data encoding for multimodal input [1]

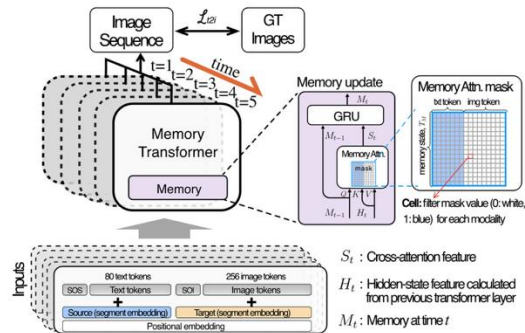


Figure 2. Detailed input configuration for the proposed transformer. As an input, we concatenate text and image tokens with two embeddings, i.e., 'SOS' (start of sentence token) and 'SOI' (start of image token), and then add positional and segment embedding. As we described in the main paper, we use memory-attention (memory attn.) mask to selectively determine which information to be propagated as a memory represented as a blue-colored box. Here, we illustrate using the 'text' information as memory content.



|       |    |                                                                 |
|-------|----|-----------------------------------------------------------------|
| Story | S1 | Loopy is angry about messy table.                               |
|       | S2 | Crong is reading a book. Pororo is thinking about next help.    |
|       | S3 | Loopy says finished in Loopy's house.                           |
|       | S4 | Loopy is finished with cleaning. Petty dusts off Loopy's hands. |
|       | S5 | Loopy is wondering who made cookies.                            |

# Multihead self-attention (MSH)

- Single set of query/key/value ( $Q, K, V$ ) projection matrix may not enough to encode information of the data
- Why not have multiple of such attention?
- Each gets its own parameterized  $Q, K, V$  functions
- Run them all in parallel, then the weighted sum the output token vectors

$$\mathbf{Z} = \begin{pmatrix} \text{attn}_1(\mathbf{t}_{\text{in}}).\mathbf{z}^T \\ \vdots \\ \text{attn}_k(\mathbf{t}_{\text{in}}).\mathbf{z}^T \end{pmatrix}$$

$$\mathbf{t}_{\text{out}}.\mathbf{z} = \mathbf{W}\mathbf{Z}$$

$$\triangleleft \mathbf{W} \in \mathbb{R}^{M_2 \times kM_1}$$

# Useful to encode the Context in language

## Buffalo buffalo Buffalo buffalo buffalo buffalo Buffalo buffalo

🌐 44 languages ▾

[Article](#) [Talk](#)

[Read](#) [View source](#) [View history](#) [Tools](#) ▾

From Wikipedia, the free encyclopedia

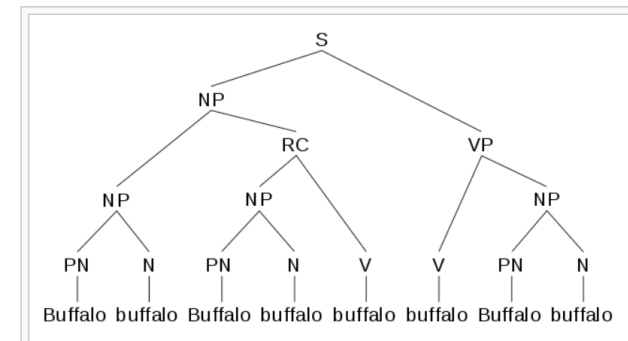


"**Buffalo buffalo Buffalo buffalo buffalo buffalo Buffalo buffalo**" is a [grammatically correct sentence](#) in [English](#) that is often presented as an example of how [homonyms](#) and [homophones](#) can be used to create complicated linguistic constructs through [lexical ambiguity](#). It has been discussed in literature in various forms since 1967, when it appeared in [Dmitri Borgmann's](#) *Beyond Language: Adventures in Word and Thought*.

The sentence employs three distinct meanings of the word *buffalo*:

- As an [attributive noun](#) (acting as an adjective) to refer to a specific place named Buffalo, such as the city of [Buffalo, New York](#);
- As the [verb](#) *to buffalo*, meaning (in [American English](#)<sup>[1]</sup>) "to bully, harass, or intimidate" or "to baffle"; and
- As a [noun](#) to refer to the animal the [buffalo](#) (often called [bison](#) outside of North America). The plural is also *buffalo*.

A semantically equivalent form preserving the original word order is: "Buffalonian bison that other Buffalonian bison bully also bully Buffalonian bison."

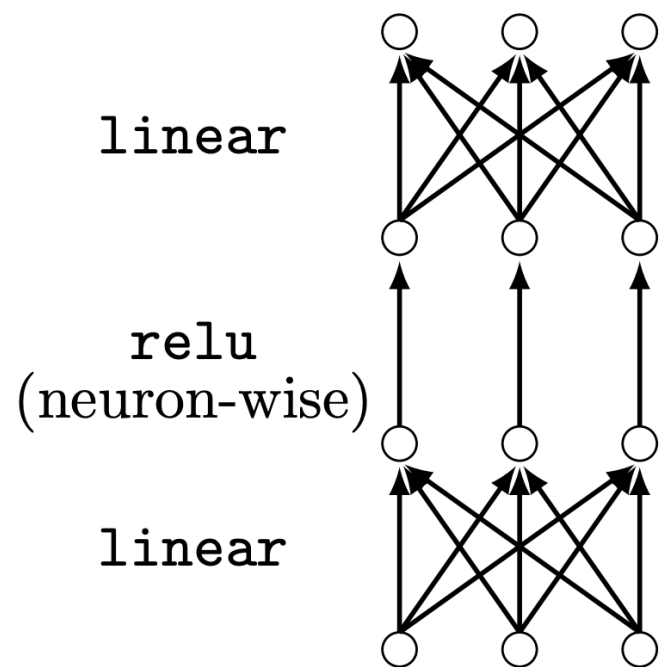


Simplified [parse tree](#)

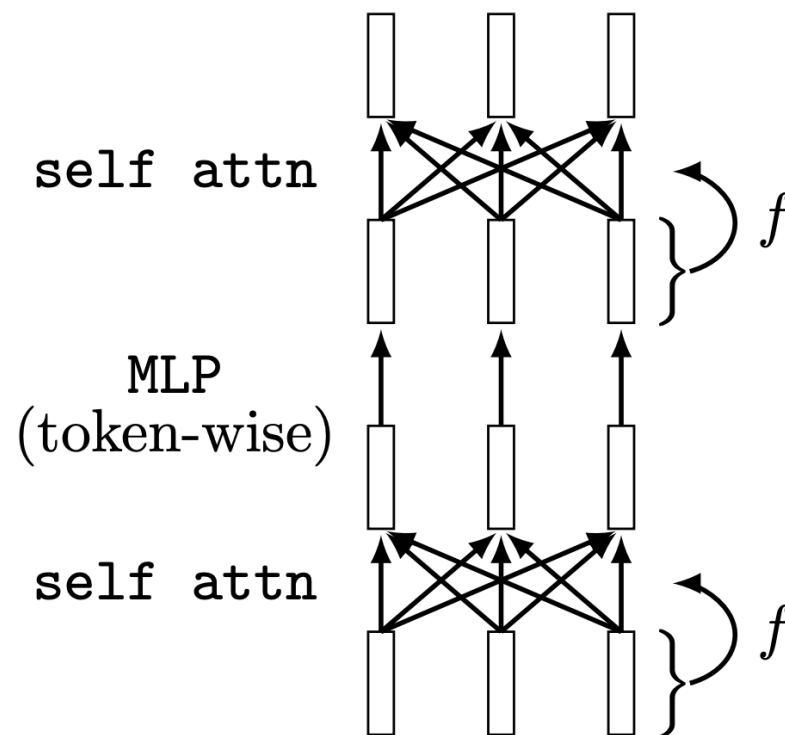
S = [sentence](#)  
NP = [noun phrase](#)  
RC = [relative clause](#)  
VP = [verb phrase](#)  
PN = [proper noun](#)  
N = [noun](#)  
V = [verb](#)

# MLP vs Transformer

MLP



Transformer (vanilla)



# Permutation equivariance

$$F_{\theta}(\text{permute}(\mathbf{T}_{\text{in}})) = \text{permute}(F_{\theta}(\mathbf{T}_{\text{in}}))$$

$$\text{attn}(\text{permute}(\mathbf{T}_{\text{in}})) = \text{permute}(\text{attn}(\mathbf{T}_{\text{in}}))$$



# Positional encoding

# If we want the model to be not permutation invariant

- **Idea**: Patches from different location should have different input
- **Implementation**: Add a value (a code) to patch input. The value should be different for each location
  - Called **positional encoding**

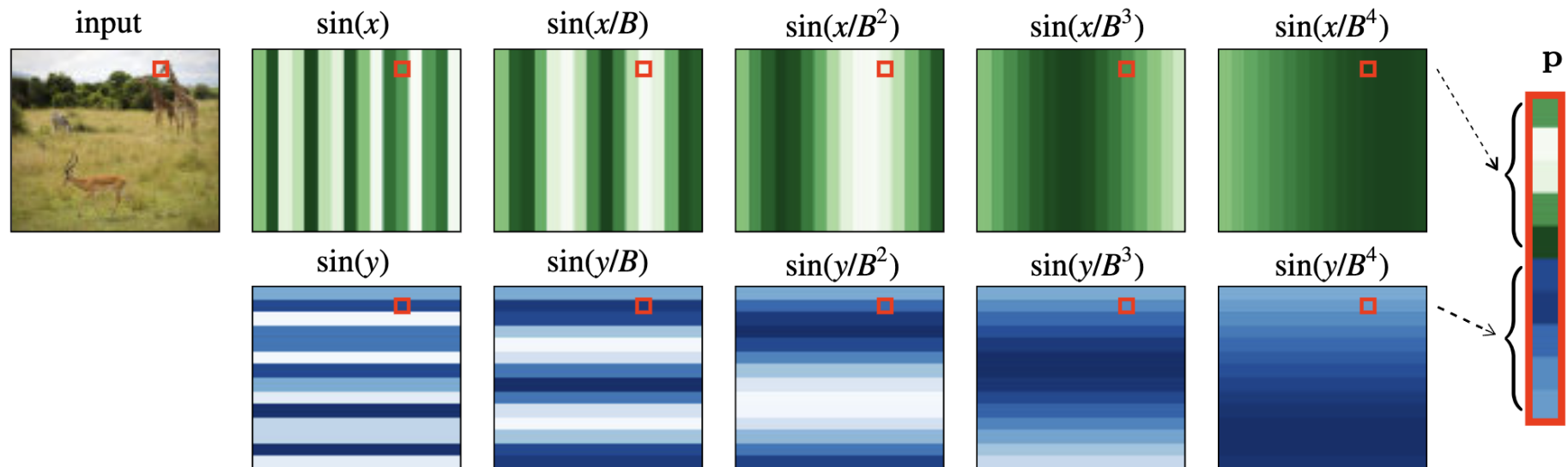
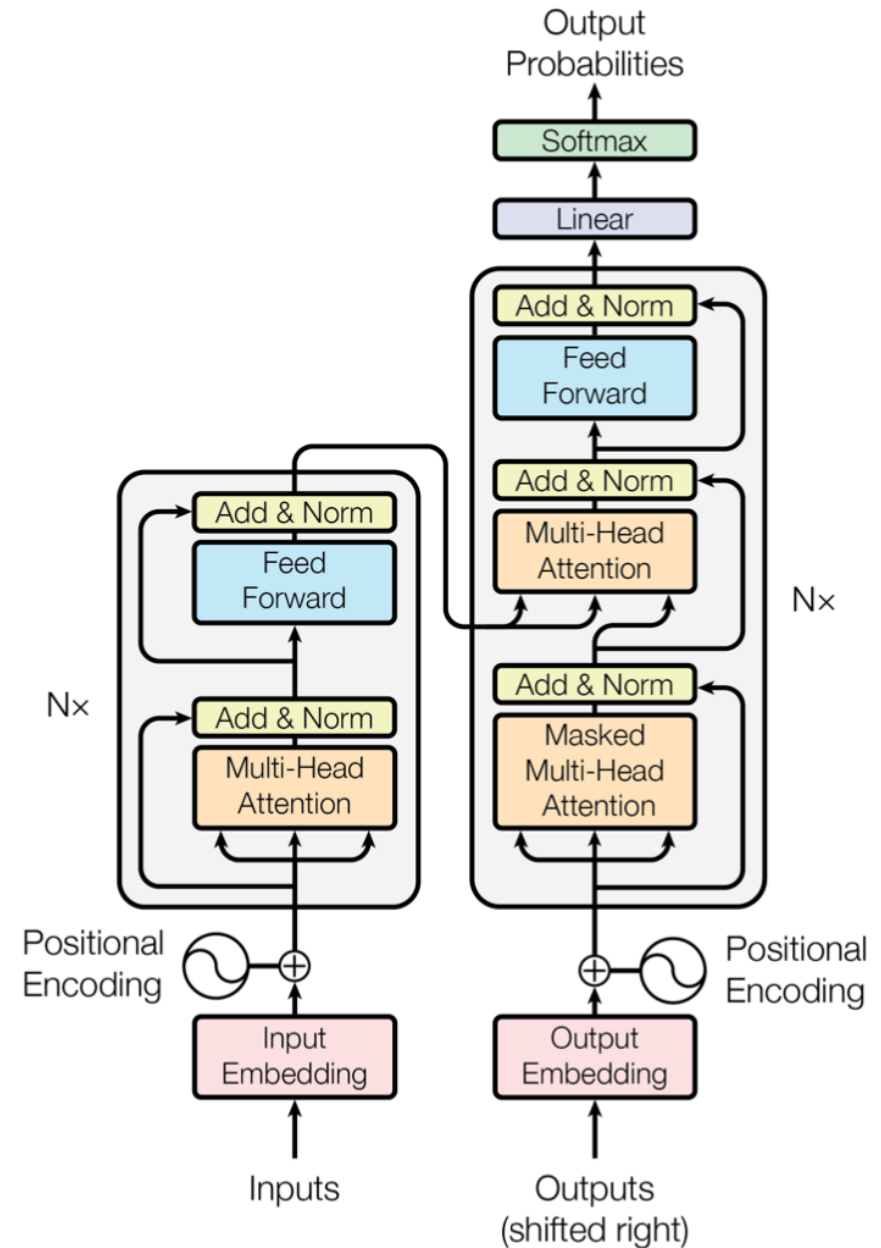


Figure 26.20: Positional codes.

# Transformer - detail

- Now, you understand the architecture of Transformers



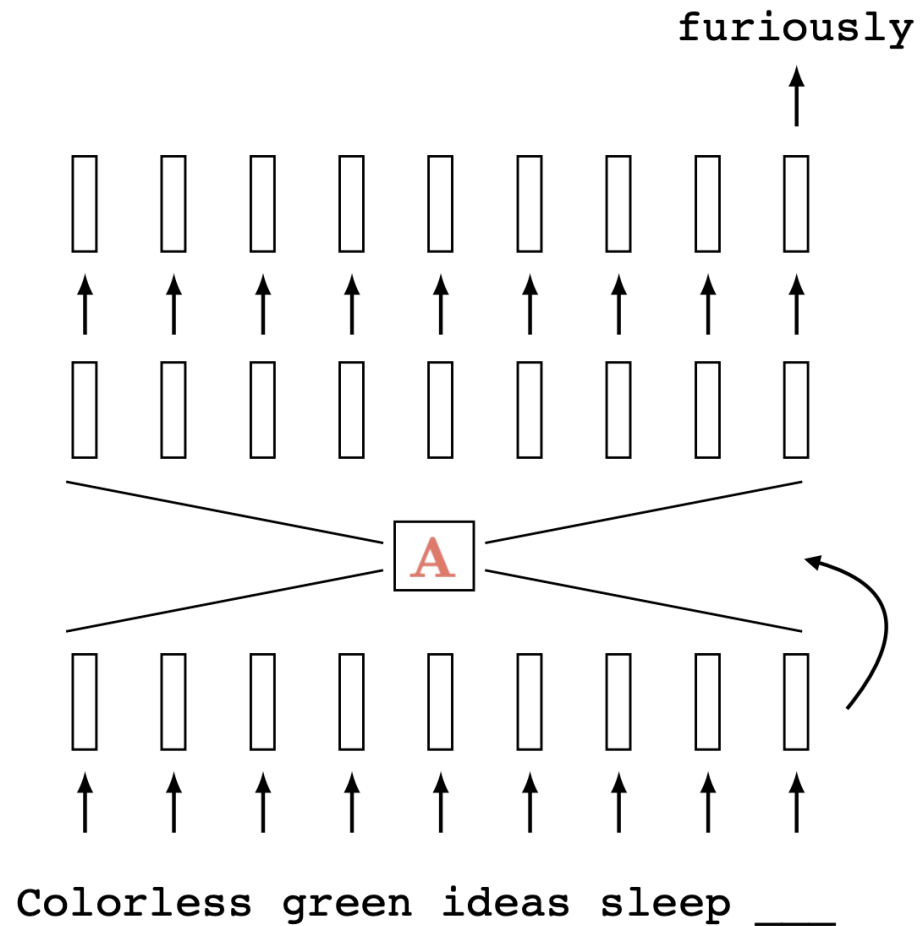
# Application

# Language model (LM)

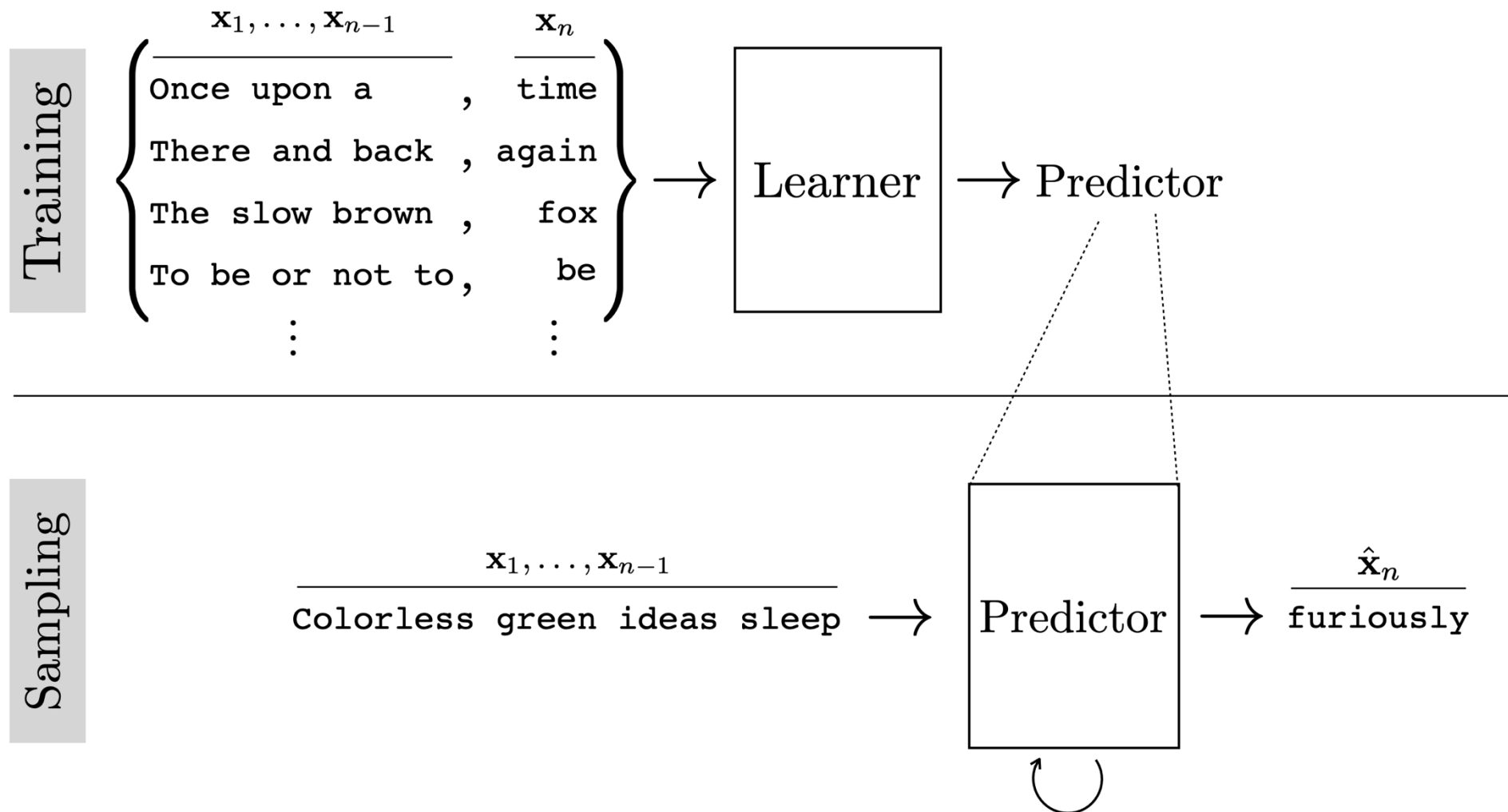
- A kind of NLP task
- A probabilistic word predictor given the sequence of previous words
  - **Assumption**: the probability of the next word in a sequence depends only on a *fixed size* window of previous words
- If the size of the LM is large, we call it **large language model (LLM)**
  - *E.g.*, GPT, Claudie, HyperClova X, *etc.*

# Autoregressive models as LMs

- An autoregressive transformer

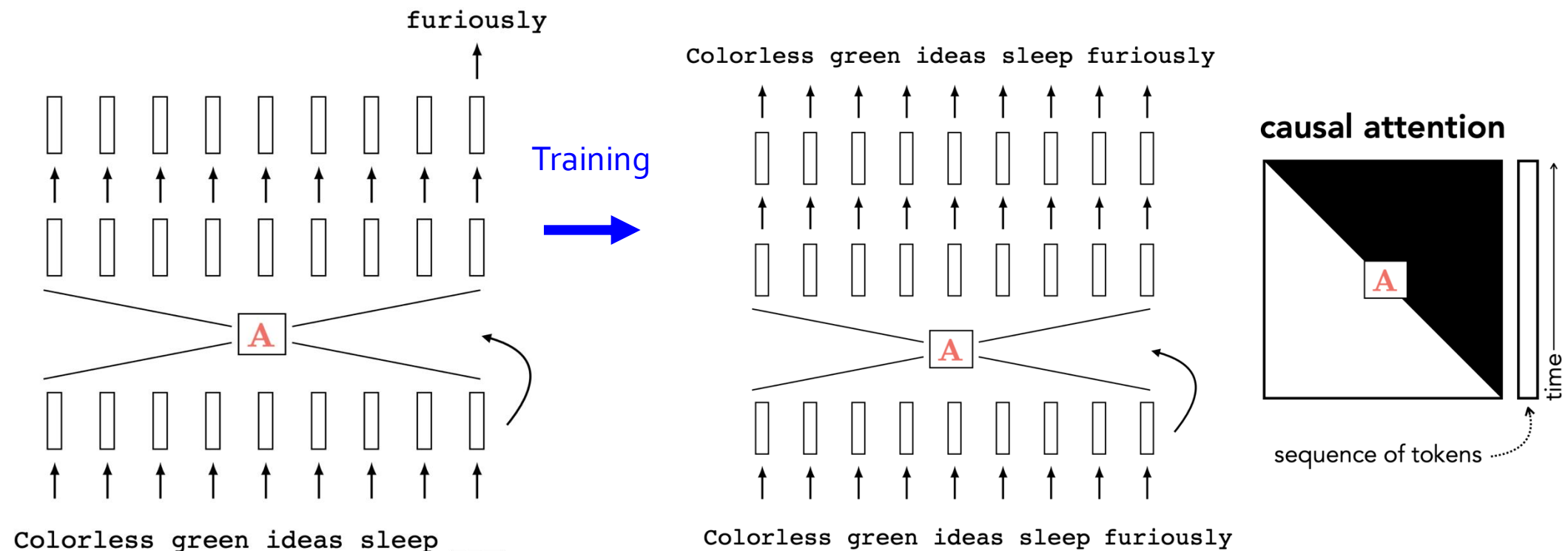


# Autoregressive models as LMs



# Training an autoregressive model with the causal attention

- Alec Radford uses Transformer for this model and name it as **Generative Pretrained Transformer (GPT)** [1]



# Play with a small version of GPT: minGPT

- <https://github.com/karpathy/minGPT>

karpathy / minGPT Public

<> Code Issues 22 Pull requests 15 Actions Projects Security Insights

master 3 branches 0 tags Go to file Add file <> Code

karpathy Merge pull request #84 from ericjang/master 7218bcf on Aug 4 93 commits

|                |                                                                         |              |
|----------------|-------------------------------------------------------------------------|--------------|
| mingpt         | Use XOR operator ^ for checking assertion `type_given XOR param...      | 4 months ago |
| projects       | refactor sequence generation into the model and match the huggingf...   | 4 months ago |
| tests          | add a refactored BPE encoder from openai. Basically I dont super tru... | 4 months ago |
| .gitignore     | tiny tweaks to printing and some function apis                          | 6 months ago |
| LICENSE        | mit license file                                                        | 2 years ago  |
| README.md      | Add setup.py to allow mingpt to be used as a third-party library        | 4 months ago |
| demo.ipynb     | refactor sequence generation into the model and match the huggingf...   | 4 months ago |
| generate.ipynb | add a refactored BPE encoder from openai. Basically I dont super tru... | 4 months ago |
| mingpt.jpg     | first commit, able to multigpu train fp32 GPTs on math and character... | 2 years ago  |
| setup.py       | Add setup.py to allow mingpt to be used as a third-party library        | 4 months ago |

README.md

## minGPT

available GPT implementations



minGPT



# Limitation of Transformers

- It cannot encode the contextual information outside of 'window'
- Computational complexity of Transformer
  - $O(n^2)$ , where  $n$  is the size of window
- There are many work to reduce the computation (e.g., improving efficiency)

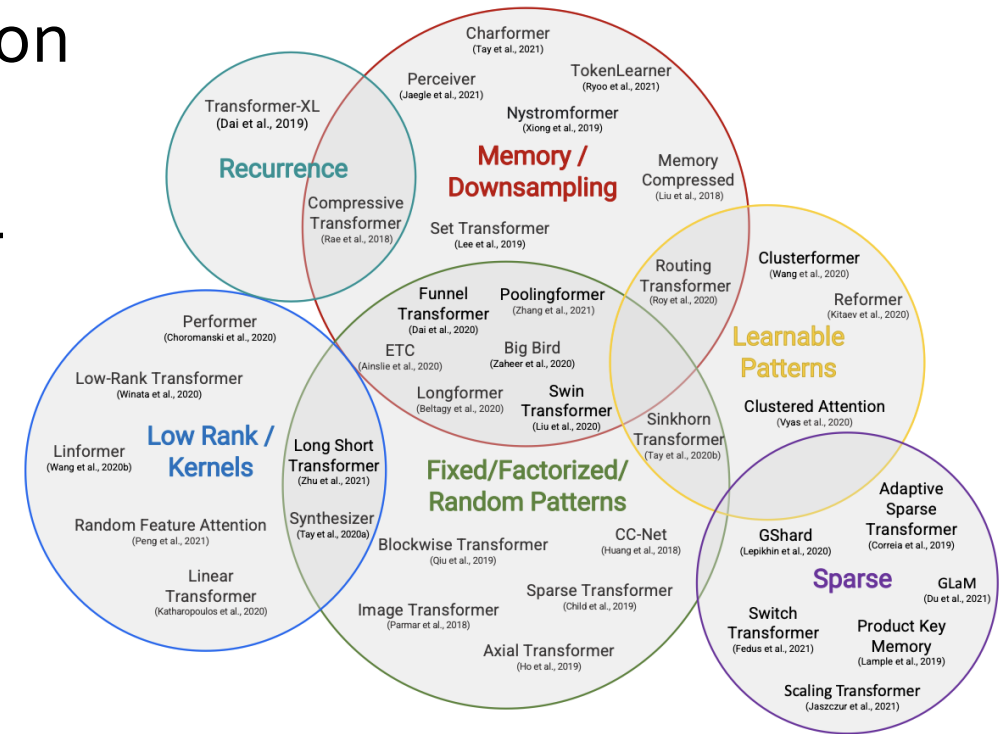
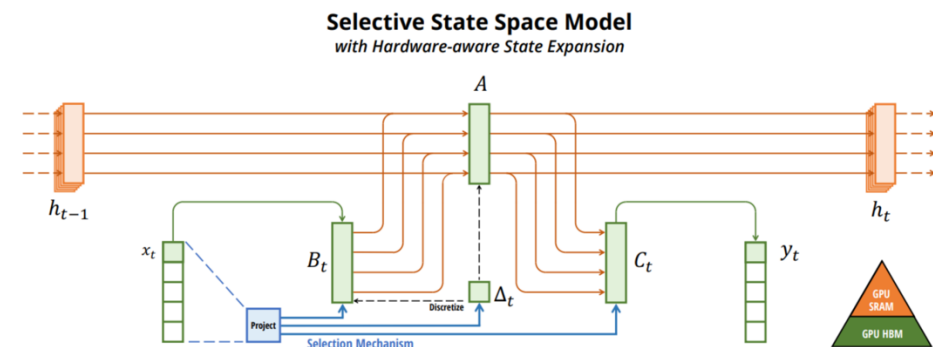


Figure 2: Taxonomy of Efficient Transformer Architectures.

# Mamba – overview

- ~~A highly venomous and fast snake~~ An efficient (fast) model for encoding sequences
- Using structured state space sequences ( $S_4$ ) model
- Two key ideas
  1. Selective mechanism ( $S_6$ )
    - Keeping a (relatively) small sized window but maintain **only relevant** tokens in the window
    - Measuring the **relevance** by something similar to the gating mechanism of RNN
  2. Hardware aware parallel scan



# Resources of Mamba

- Paper: <https://arxiv.org/pdf/2312.00752>
- Official implementation: <https://github.com/state-spaces/mamba>
- Variants
  - Vision Mamba (Vim)
  - Jamba
    - Taking 256k token window (c.f., ChatGPT-4o and o1 has token size of 128k)

# Summary

- Three key ideas of Transformers
  - Tokens
  - Attention
  - Position encoding
- Application: GPT, ViT
- Computationally efficient alternative: Mamba and *etc.*